SELECTED WORKS / 웹·백엔드

이커머스

주문부터 반품까지, 처리 결과를 끝까지 확인하는 쇼핑몰

주문·결제·환불·반품의 경합과 결과 불명 복구를 다룬 단일 판매자 쇼핑몰입니다.

https://github.com/MaoEmong/commerce-ai-portfolio

웹 상세 소개 보기

Java · Spring Boot · React · TypeScript · PostgreSQL

서비스 범위	의류·잡화 단일 판매자 쇼핑몰 / 고객·관리자
백엔드 구조	Spring Boot 모듈형 단일 앱 + PostgreSQL
정합성 중심 흐름	SKU 재고 예약·멱등 요청·결제 복구·운영 확인 큐
외부 연동 상태	Fake PG / local·test AI 대역, 실모델 어댑터 미구현

01 / CONTEXT

어떤 문제에서 시작했나요?

쇼핑몰의 어려움은 결제 성공 화면보다 성공 여부를 모르는 순간에 드러납니다. 사용자가 주문 버튼을 누른 뒤 응답을 받지 못해 다시 눌렀을 때 주문이 늘어나거나, 취소된 주문에 늦은 결제 승인이 도착해 재고가 다시 차감될 수 있습니다. 이 프로젝트는 상품 탐색부터 반품까지 구현하면서 요청과 업무 상태를 분리하고, 불확실한 결과를 다시 확인하는 복구 경로를 핵심으로 다뤘습니다.

관리자에게도 예외를 처리할 화면이 필요합니다. 환불은 성공했지만 실물 반품은 아직 도착하지 않을 수 있고, 재고를 복원할 시점은 돈의 흐름과 다를 수 있습니다. 따라서 고객 구매 흐름과 운영 확인 큐를 함께 설계했습니다. AI 기능 역시 상품을 임의로 추천·수정하는 권한으로 확장하기보다 기존 검색 조건을 제안하고 승인 전 설명 초안을 만드는 보조 경로로 제한했습니다.

02 / PRODUCT FLOW

기능과 사용자 흐름

고객의 구매 흐름

검색·조건별 탐색, 색상·크기별 상품 선택, 장바구니와 주문·취소·반품을 연결합니다.

관리자의 운영 흐름

상품·재고·입고·배송·반품 승인과 결과 확인이 필요한 결제·환불 요청을 관리합니다.

AI 기능의 적용 경계

자연어 상품 검색과 상품 설명 초안을 제공합니다. 현재는 모의 응답이며 설명 초안은 담당자가 승인합니다.

한 번의 사용을 따라가 보면

01 1. 조건 탐색과 SKU 선택

카테고리·검색어·가격·색상·크기·재고 조건으로 상품을 좁힙니다. 상세 화면에서는 실제 존재하는 색상·크기 조합만 선택할 수 있고 품절 조합을 차단합니다. 자연어 탐색도 같은 검색 규칙을 이용합니다.

02 2. 장바구니에서 재고 예약으로

장바구니는 구매 불가 사유를 표시하고 선택한 항목으로 주문서를 만듭니다. 주문 생성은 멱등 키와 함께 요청하며 서버가 SKU별 재고를 예약합니다. 결과를 받지 못한 요청은 원본 본문과 키를 보관해 같은 요청으로 재확인합니다.

03 3. 결제·취소·배송의 상태 전이

Fake PG의 승인·거절·결과 불명 경로를 처리합니다. 결제 전 취소는 예약을 해제하고 결제 후 출고 전 취소는 환불 의무를 생성합니다. 관리자의 출고와 고객의 취소가 겹치면 주문 행 잠금으로 전이를 조정합니다.

04 4. 반품과 운영 예외의 마무리

배송 완료 후 고객이 반품을 신청하면 관리자가 승인·거절하고 실물 수령을 확인합니다. 환불과 재고 복원은 각각 기록합니다. 자동 복구 횟수를 넘거나 대사 결과가 어긋난 결제·환불은 운영 확인 큐에서 재조회·수동 처리합니다.

05 5. 사람이 승인하는 상품 설명

관리자는 상품 정보로 생성한 설명 초안을 검토·수정한 뒤 승인합니다. 승인 전에는 실제 상품에 반영되지 않으며 기존 상품 쓰기 경로로 적용해 별도 AI 변경 통로가 생기지 않도록 구성했습니다.

03 / ARCHITECTURE

구조와 책임을 나눈 방식

01 단일 앱 안의 검증 가능한 모듈 경계

회원·상품·재고·장바구니·주문·결제·AI·운영 기능을 Spring Boot 앱 안의 모듈로 나눴습니다. 다른 모듈은 공개 api 패키지를 통해 호출하고 internal·web에 직접 의존하지 않습니다. ArchUnit은 이 경계와 AI의 CommandService 의존 금지, Fake PG 어댑터 직접 참조 금지를 검사합니다.

02 주문과 결제 시도의 분리

주문 생성 시 SKU 재고를 15분 예약하고 결제 확정·만료·취소에 따라 확정하거나 해제합니다. 결제는 주문 상태와 별도의 시도·복구 상태로 관리합니다. 환불 의무와 환불 시도도 분리해 외부 호출 전에 기록을 커밋하고 트랜잭션 밖에서 PG를 호출합니다.

03 DB 권한과 검색용 읽기 경로

업무 app과 fakepg 스키마에 런타임·마이그레이션 계정을 각각 분리했습니다. 일반 실행 계정은 DML만 허용하고 서로의 스키마 접근을 차단합니다. 상품 검색은 같은 트랜잭션에서 갱신한 정규화 projection과 재고 읽기 전용 뷰를 이용해 일반 검색과 AI 검색이 같은 조건을 사용합니다.

04 검증된 조건으로만 이어지는 AI

LlmClient 포트가 필터 추출·답변·상품 초안을 나누어 반환하고 서버가 JSON·허용값·카드 참조를 검증합니다. 답변이 검증에 실패하면 상품 카드만 보여 줍니다. 기본 설정은 disabled, local·test는 fake를 사용할 수 있으며 실제 모델 어댑터는 아직 없습니다.

04 / ENGINEERING CASES

구현하면서 선택한 것

CASE 01 / 응답 유실을 중복 주문으로 만들지 않기

마주한 문제

네트워크 오류는 서버의 처리 실패와 같지 않습니다. 새 키로 주문·취소를 재전송하면 이미 끝난 처리가 다시 실행될 수 있고, 브라우저 새로고침은 요청의 맥락을 잃게 만듭니다.

선택과 구현

쓰기 요청은 Idempotency-Key와 본문을 묶어 관리합니다. 같은 키·본문은 기존 결과를 돌려주고 다른 본문은 거부합니다. 프런트는 회원별 sessionStorage에 결과 불명 요청을 보관하고 같은 키로 확인하며 로그아웃 경계에서도 미확인 작업을 알립니다.

남은 비용과 한계

클라이언트와 서버가 요청 보관·재확인 수명까지 함께 관리해야 합니다. 재시도 버튼만 추가하는 방식보다 복잡하지만 사용자가 실패 여부를 추측해 새 주문을 만드는 상황을 줄이는 방향입니다.

설계 요약 — 멱등성과 결과 불명 복구

CASE 02 / 늦은 승인과 복구 작업의 경합 제어

마주한 문제

요청 처리 중 lease가 만료되어 복구 작업이 같은 결제를 가져가면 오래된 작업의 응답이 최신 상태를 덮을 수 있습니다. 주문 만료 뒤 결제가 승인되는 상황도 별도로 정리해야 합니다.

선택과 구현

결제 시도에 상태·lease·fencing token을 저장합니다. 복구 작업이 claim할 때 토큰을 올리고 결과 반영에 기대 상태·토큰 조건을 사용합니다. 늦은 승인은 해당 시도에 대한 승인 취소·정리 환불로 연결하고 이미 만료·취소된 주문의 예약·재고는 되돌리지 않습니다.

남은 비용과 한계

결제 시도와 주문·환불·대사의 상태가 늘어나며 마지막 자동 복구 실패를 처리할 운영 큐가 필요합니다. 현재는 같은 DB의 Fake PG로 구조를 재현하며 실제 PG의 장애·콜백 검증은 별도로 필요합니다.

PaymentAttemptRepository — claim·상태·토큰 조건

CASE 03 / AI가 업무 쓰기 권한을 우회하지 않도록

마주한 문제

생성형 모델의 필터·설명은 잘못된 값이나 근거 없는 주장을 포함할 수 있습니다. AI 모듈에서 주문·상품 쓰기 서비스를 직접 호출하면 검증·승인 흐름을 우회할 여지가 생깁니다.

선택과 구현

AI 모듈의 CommandService 의존을 ArchUnit으로 금지하고, 필터는 기존 검색 경로로, 상품 초안은 사람 승인 뒤 기존 쓰기 경로로 보냅니다. LlmClient는 신뢰하지 않는 JSON 문자열을 반환하며 서버 검증을 통과하지 못한 답변은 노출하지 않습니다.

남은 비용과 한계

검증을 위해 기능을 좁히고 모델을 교체할 포트를 먼저 정의했습니다. 현재는 실모델 어댑터가 없어 전송 정책·호출 상한을 실제 모델에 적용하고 품질·비용·지연을 평가하는 작업이 남아 있습니다.

ArchitectureTest — 모듈·AI 쓰기 의존 검사

05 / ROLE & CONTRIBUTION

맡은 역할과 기여

고객의 구매 흐름과 관리자의 예외 처리 흐름을 함께 다루도록 프로젝트 범위를 정했습니다. 특히 응답을 받지 못한 주문, 늦게 도착한 결제 승인, 환불과 반품의 서로 다른 완료 시점을 주요 요구사항으로 잡고, 픽셀 오피스에서 Claude Code·Codex의 구현·검증 작업을 관리했습니다.

화면 흐름·ERD·API·상태 전이 초안부터 구현과 테스트·리뷰까지 AI 코딩 도구를 활용했습니다. 테스트 결과와 실제 화면, 독립 리뷰를 대조해 지적된 부분을 수정했습니다. 개발에 사용한 AI와 제품 안의 자연어 탐색 기능은 별도로 다뤘으며, 시연용 상품 이미지도 AI로 제작했습니다.

06 / EVIDENCE & LIMITS

무엇을 확인했고, 무엇이 남았나요?

프로젝트 검증 당시의 실행 기록과 코드에 구성한 테스트 항목을 정리했습니다. 각 항목에 실행 조건과 추가 확인이 필요한 범위를 함께 적었습니다.

프런트 공개본 검사

공개본 점검에서 npm ci·typecheck·Vitest·build가 통과했습니다. Vitest 결과는 21개 파일, 319개 테스트입니다.

프런트의 타입·단위 테스트와 빌드를 확인한 결과입니다. 백엔드와 외부 결제까지 연결한 통합 결과는 아래 항목과 나누어 정리했습니다.

검증 근거 보기

백엔드 컴파일·단위 검사

공개본에서 compileJava·compileTestJava와 컨테이너 없이 실행하는 단위 검사 40개가 통과했습니다.

ArchitectureTest·AiConfigTest·QueryMaskerBoundaryTest 대상입니다. Testcontainers 통합 검사 33개 클래스는 공개본 기준 재실행이 남아 있습니다.

검증 근거 보기

빈 DB 기동 확인

새 PostgreSQL 17.11 DB에서 readiness UP, Flyway app 12/12·fakepg 2/2 적용, 상품 12개와 최상위 카테고리 5개 조회를 확인했습니다.

docker run·java -jar로 기동한 결과입니다. Windows PowerShell 실행 스크립트 전체 경로는 별도 재검증이 필요합니다.

검증 근거 보기

현재 범위와 남은 과제

• 실제 PG·웹훅을 연결하지 않았습니다. 결제·환불은 같은 DB의 별도 스키마와 계정으로 구성한 Fake PG입니다.

• 제품 AI는 local·test 대역으로 시연하며 기본은 비활성입니다. 실모델 어댑터·품질·비용·지연 평가와 서버 통합 검사 공개본 재실행이 남아 있습니다.

• 외부 배포·운영 트래픽·부하 측정 이력은 없습니다. 제공 실행 스크립트는 Windows·PowerShell 7을 전제로 합니다.

• 상품 이미지는 AI로 만든 시연 예시입니다. 공개본 Flyway SQL은 빈 DB 초기화를 기준으로 구성했습니다. 기존 DB에 적용하려면 해당 환경의 마이그레이션 이력을 먼저 검토해야 합니다.

다음에 보완할 것

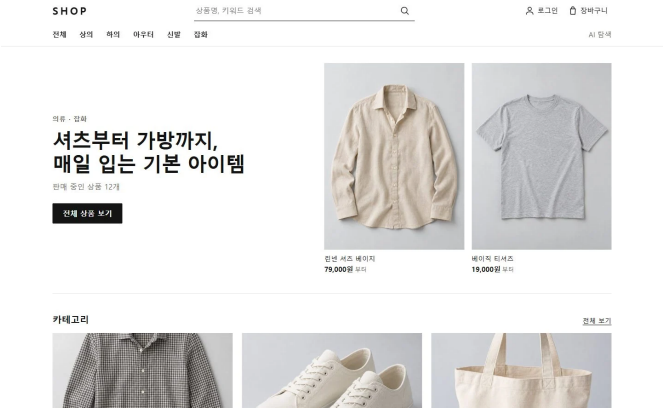
• 공개본의 전체 Testcontainers 통합 검사를 재실행하고 결제·환불 경합과 DB 역할 분리를 동일 구성에서 확인합니다.

• 실제 PG 어댑터·콜백을 별도로 구현해 지연 승인·재전송·대사 시나리오를 검증하고, 실모델 연결은 품질·안전·비용 평가와 함께 진행합니다.

07 / SCREENS

프로젝트 화면

공개본을 로컬에서 실행한 고객 화면입니다. 상품 이미지는 AI로 만든 소개용 예시이며 실제 상품 사진이 아닙니다.



이커머스 · 고객 홈 화면



이커머스 · 상품 상세와 선택 항목



이커머스 · 모바일 홈 화면

08 / SOURCE MATERIAL

코드와 개발 기록

아래 공개 자료에서 구현 범위와 설계 판단을 확인할 수 있습니다. 링크를 누르면 원문으로 이동합니다.

01 프로젝트 README와 공개본 검증 범위

https://github.com/MaoEmong/commerce-ai-portfolio/blob/main/README.md

02 주문·결제·검색·AI 설계 요약

https://github.com/MaoEmong/commerce-ai-portfolio/blob/main/docs/DESIGN.md

03 결제 시도와 복구 조건 실제 소스

https://github.com/MaoEmong/commerce-ai-portfolio/blob/main/backend/src/main/java/com/shop/payment/internal/PaymentAttemptRepository.java

04 ArchUnit 모듈 의존 규칙

https://github.com/MaoEmong/commerce-ai-portfolio/blob/main/backend/src/test/java/com/shop/ArchitectureTest.java

05 LlmClient — 대역·비활성 및 실모델 미구현

https://github.com/MaoEmong/commerce-ai-portfolio/blob/main/backend/src/main/java/com/shop/ai/internal/LlmClient.java